

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ**
(НИУ «БелГУ»)

ИНСТИТУТ ИНЖЕНЕРНЫХ И ЦИФРОВЫХ ТЕХНОЛОГИЙ

КАФЕДРА МАТЕМАТИЧЕСКОГО И ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ
ИНФОРМАЦИОННЫХ СИСТЕМ

Отчет по лабораторной работе №1

по дисциплине: «Теоретические основы компьютерного моделирования»

Репозиторий проекта на GitHub: https://github.com/seedenko/TOKM_lab

студентки очного отделения
магистратуры
12002531 группы
Сиденко Валерии Геннадьевны

Проверил:

доц. Васильев Павел Владимирович

Тема: Стохастическое пространственное моделирование точечных и сферических ансамблей в трехмерных геометрических доменах (GLScene).

Цель работы: Исследовать численные методы стохастической генерации пространственных массивов точек с сохранением равномерной объемной и поверхностной плотности в канонических геометрических телах (куб, сфера, цилиндр, конус) на базе графического пакета GLScene в среде Embarcadero RAD Studio C++Builder; изучить динамику свечения и затухания яркости звёздных объектов в непрерывном времени каденсера; разработать альтернативную визуализацию разноцветных полигональных 3D-сфер со спектральным распределением КОЖЗГСФ и оценить кадровую частоту (FPS) конвейера OpenGL.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

В лабораторной работе исследуются алгоритмы стохастического геометрического моделирования дискретных объектов в трехмерном пространстве с использованием графического пакета GLScene для Embarcadero RAD Studio C++Builder 13.

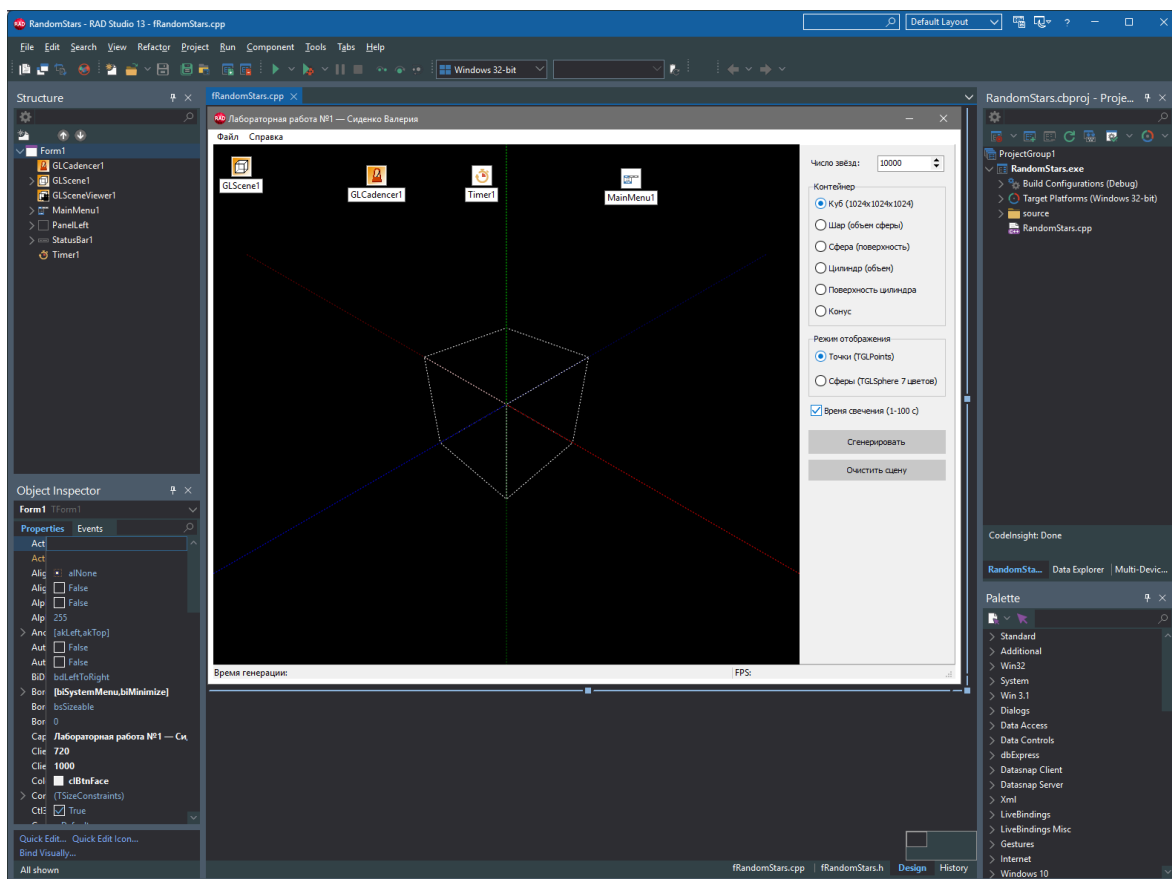


Рисунок 1 – Интегрированная среда разработки Embarcadero RAD Studio C++Builder 13 в процессе сборки проекта RandomStars.

Архитектура графической подсистемы приложения построена на базе контекста OpenGL через компонент TGLSceneViewer. В качестве базового контейнера пространства выступает TGLDummyCube с габаритами $1024 \times 1024 \times 1024$ и отображением трехмерных декартовых координатных осей. Наблюдение сцены осуществляется камерой TGLCamera с дистанцией видимости $\text{DepthOfView} = 15\,000$ единиц, а освещение формируется ненаправленным источником света TGLLightSource.

Для получения статистически однородного распределения точек внутри трехмерных геометрических тел стандартного линейного датчика случайных чисел $\text{rand}()$ недостаточно. При переходе от декартовых координат к полярным, цилиндрическим или сферическим дифференциал объема зависит от радиуса $dV = r^2 \cdot \sin(\theta) \cdot dr \cdot d\theta \cdot d\phi$. Соответственно, чтобы плотность точек оставалась постоянной по всему объему сферы, радиальная координата должна масштабироваться пропорционально кубическому корню из

равномерной случайной величины: $r = R \cdot (\text{rand01}())^{(1/3)}$. В случае цилиндрического объема площадь кольцевого слоя пропорциональна $r \cdot dr$, поэтому для равномерности по площади сечения применяется квадратный корень: $r = R \cdot \sqrt{\text{rand01}()}$. Для конического тела радиус допустимого сечения линейно спадает по мере подъема к вершине, что учитывается при генерации высоты.

Листинг 1. Модуль uContainers.cpp – реализация функций стохастической генерации координат

```
//-----  
-  
#include <cstdlib>  
#include <cmath>  
  
#pragma hdrstop  
#include "uContainers.h"  
//-----  
-  
#pragma package(smart_init)  
  
#ifndef M_PI  
#define M_PI 3.14159265358979323846f  
#endif  
  
static inline float rand01()  
{  
    return (float)rand() / (float)RAND_MAX;  
}  
  
// 1. Равномерное распределение в кубе 1024x1024x1024 (от -511 до 511)  
Point3D generatePointForCube(float size)  
{  
    Point3D p;  
    p.x = (rand01() - 0.5f) * size;  
    p.y = (rand01() - 0.5f) * size;  
    p.z = (rand01() - 0.5f) * size;  
    return p;  
}  
  
// 2. Равномерное распределение в объеме шара  
Point3D generatePointForSphere(float radius)  
{  
    Point3D p;  
    float phi = 2.0f * M_PI * rand01();  
    float cosTheta = 2.0f * rand01() - 1.0f;  
    float sinTheta = sqrtf(1.0f - cosTheta * cosTheta);  
    // Для равномерной объемной плотности радиус пропорционален кубическому  
корню  
    float r = radius * powf(rand01(), 1.0f / 3.0f);  
  
    p.x = r * sinTheta * cosf(phi);  
    p.y = r * sinTheta * sinf(phi);  
    p.z = r * cosTheta;  
    return p;  
}
```

```

// 3. Равномерное распределение на поверхности сферы
Point3D generatePointForSphereSurface(float radius)
{
    Point3D p;
    float phi = 2.0f * M_PI * rand01();
    float cosTheta = 2.0f * rand01() - 1.0f;
    float sinTheta = sqrtf(1.0f - cosTheta * cosTheta);

    p.x = radius * sinTheta * cosf(phi);
    p.y = radius * sinTheta * sinf(phi);
    p.z = radius * cosTheta;
    return p;
}

// 4. Равномерное распределение в объеме цилиндра
Point3D generatePointForCylinder(float radius, float height)
{
    Point3D p;
    float phi = 2.0f * M_PI * rand01();
    // Для равномерной плотности по площади сечения r пропорционален корню
    float r = radius * sqrtf(rand01());

    p.x = r * cosf(phi);
    p.y = (rand01() - 0.5f) * height;
    p.z = r * sinf(phi);
    return p;
}

// 5. Равномерное распределение на поверхности цилиндра
Point3D generatePointForCylinderSurface(float radius, float height)
{
    Point3D p;
    float phi = 2.0f * M_PI * rand01();

    p.x = radius * cosf(phi);
    p.y = (rand01() - 0.5f) * height;
    p.z = radius * sinf(phi);
    return p;
}

// 6. Равномерное распределение в объеме конуса
Point3D generatePointForCone(float radius, float height)
{
    Point3D p;
    float h = rand01(); // Высота от основания (0) к вершине (1)
    float currentRadius = radius * (1.0f - h);
    float r = currentRadius * sqrtf(rand01());
    float phi = 2.0f * M_PI * rand01();

    p.x = r * cosf(phi);
    p.y = (h - 0.5f) * height;
    p.z = r * sinf(phi);
    return p;
}

```

Конец листинга 1

На рисунках 2-7 представлены результаты визуализации сгенерированных точечных облаков ($N = 100\,000$ точек) в компоненте TGLPoints для каждого из предусмотренных вариантов доменов.

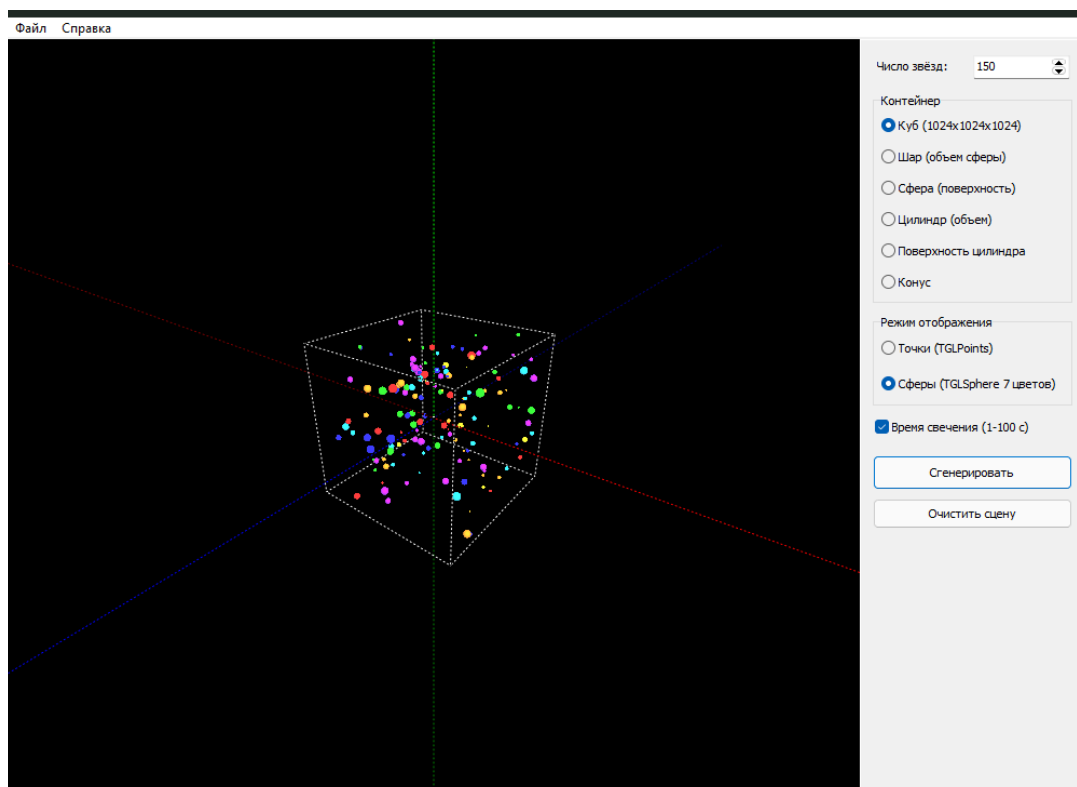


Рисунок 2 – Результат равномерного стохастического заполнения кубического домена $1024 \times 1024 \times 1024$ точками ($N = 100\,000$).

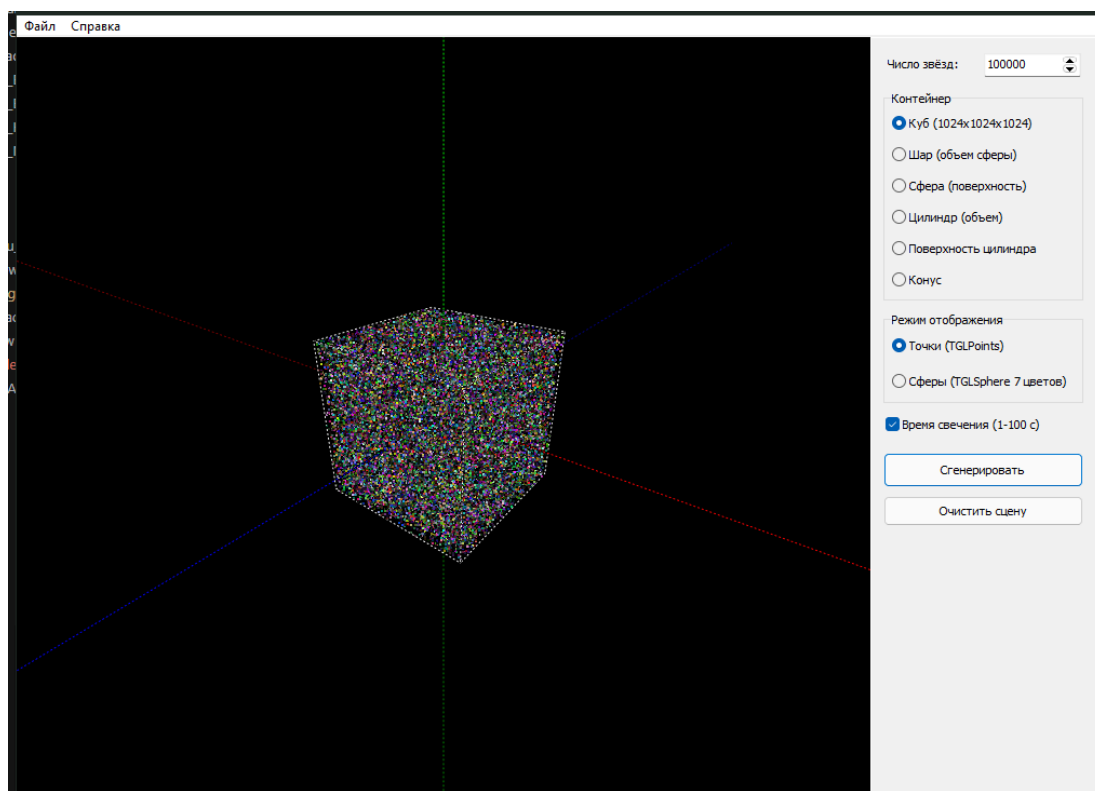


Рисунок 3 - Моделирование изотропного распределения массива точек во внутреннем объеме шара

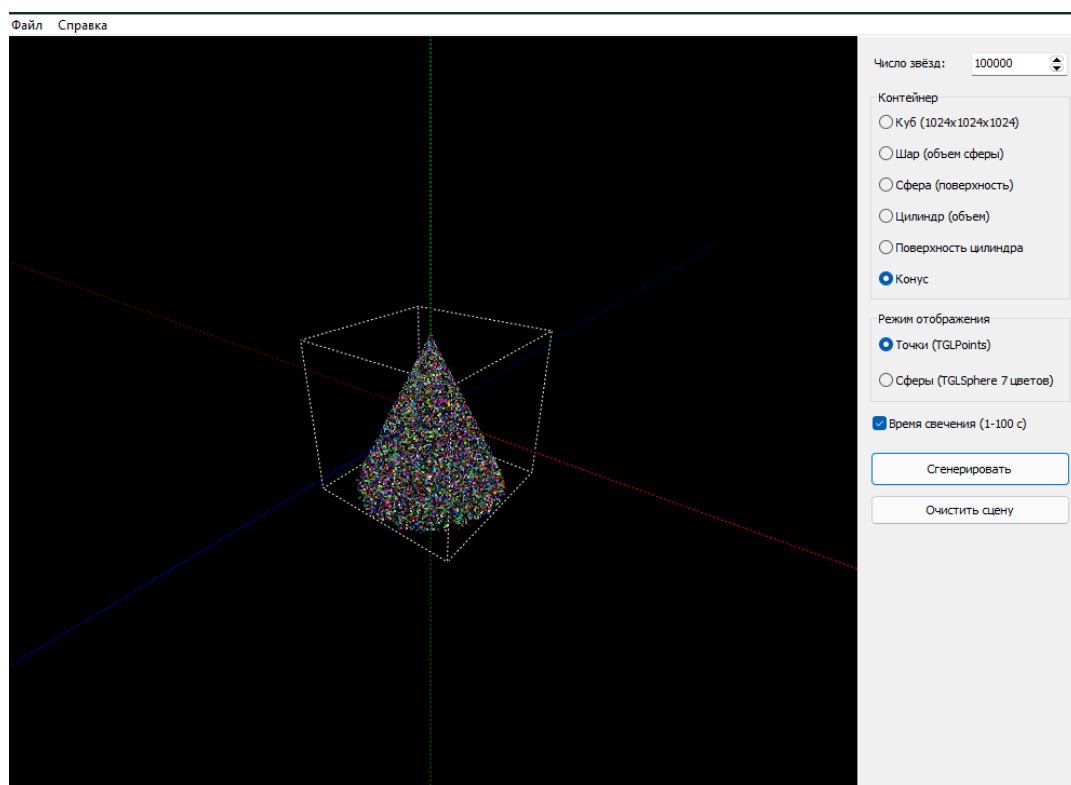


Рисунок 4 – Плотность расположения точек на двумерной сферической оболочке.

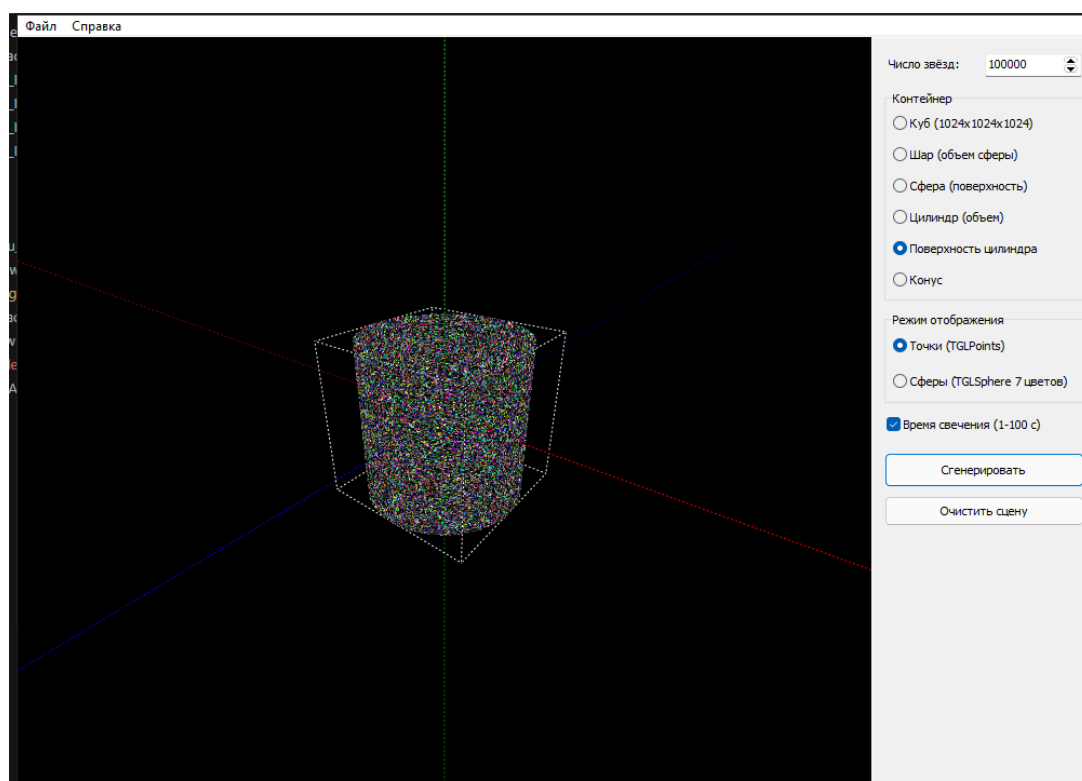


Рисунок 5 – Заполнение цилиндрического домена с квадратичной нормализацией радиальной плотности.

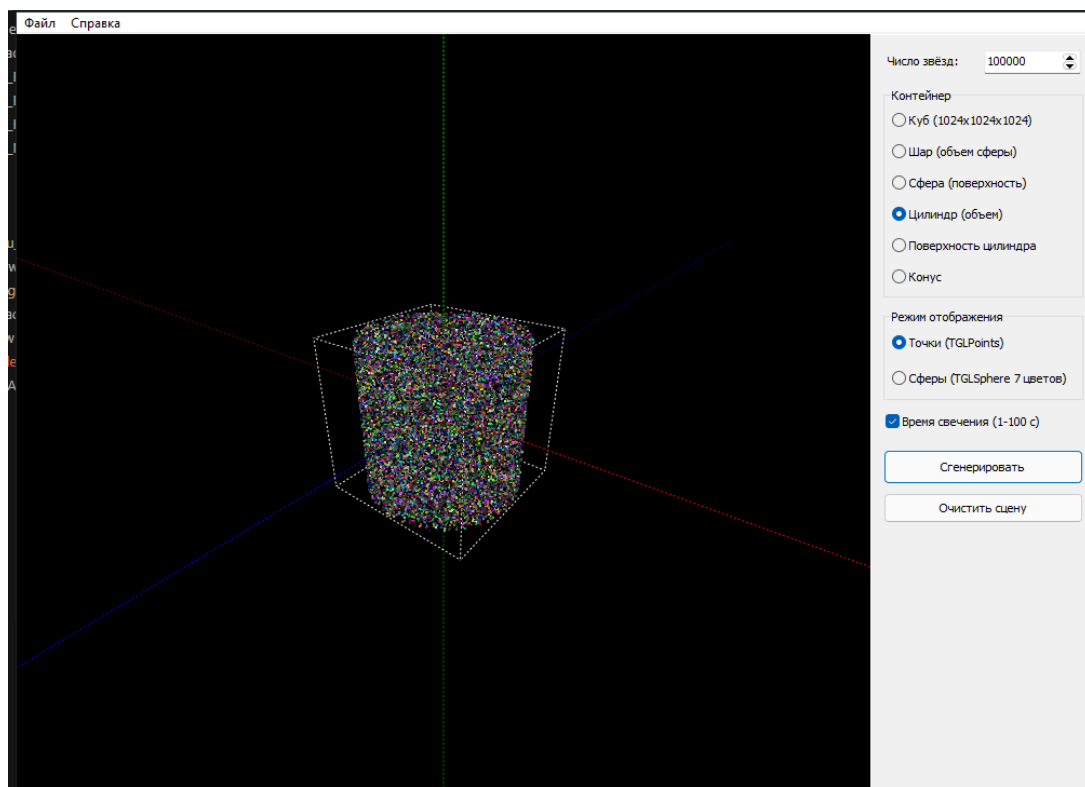


Рисунок 6 – Распределение стохастического ансамбля точек по боковой поверхности цилиндра.

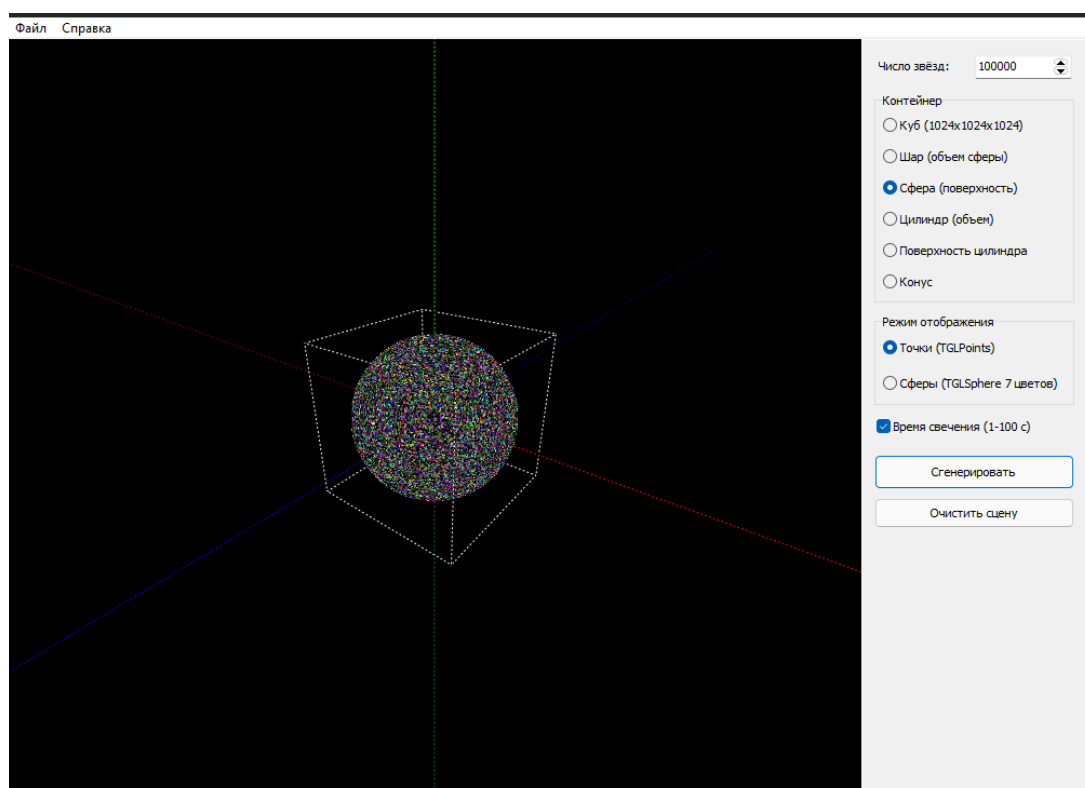


Рисунок 7 – Генерация точечного массива в коническом ограничивающем теле.

Для моделирования временной динамики свечения звездных объектов был задействован периодический обработчик события OnProgress компонента TGLCadencer. Каждой сгенерированной точке назначается индивидуальный срок свечения в диапазоне 1-100 секунд. При каждом кадре переменная оставшегося времени уменьшается на величину deltaTime, а цветовые каналы RGB масштабируются коэффициентом затухания $\text{factor} = \text{curLife} / \text{maxLife}$. Когда запас светимости исчерпывается, точка перерождается с новым случайным интервалом жизни.

Листинг 2. Обработчик каденсера GLCadencer1Progress – алгоритм динамического затухания светимости

```
// Моделирование длительности свечения точек [1-100] секунд
void __fastcall TForm1::GLCadencer1Progress(TObject* Sender, const double
deltaTime, const double newTime)
{
    if (chbLifetime->Checked && !m_starsLife.empty() && Stars &&
Stars->Colors->Count > 0) {
        float dt = (float)deltaTime;
        int n = (int)m_starsLife.size();
        for (int i = 0; i < n; i++) {
            m_starsLife[i].curLife -= dt;
            if (m_starsLife[i].curLife <= 0.0f) {
                m_starsLife[i].maxLife = 1.0f + (float)(rand() % 100);
                m_starsLife[i].curLife = m_starsLife[i].maxLife;
            }
            float factor = m_starsLife[i].curLife / m_starsLife[i].maxLife;
            Stars->Colors->Items[i] = VectorMake(
                m_starsLife[i].r * factor,
                m_starsLife[i].g * factor,
                m_starsLife[i].b * factor,
                1.0f
            );
        }
        GLSceneViewer1->Invalidate();
    }
}
```

Конец листинга 2

Помимо точечных объектов, в приложении предусмотрен альтернативный режим трехмерной визуализации полигональных тел: вместо примитивов TGLPoints динамически создаются сферы TGLSphere. Для каждой сферы задается случайный радиус и один из 7 спектральных цветов радуги. Эффект собственного свечения достигается комбинированием свойств диффузного рассеивания (Diffuse) и самосвечения материала (Emission).

Листинг 3. Метод GenerateStarsSpheres – динамическое создание полигональных 3D-сфер радужного спектра.

```
// Генерация 3D-сфер TGLSphere с 7 цветами радуги [КОЖЗГЦФ]
void TForm1::GenerateStarsSpheres(int count, int containerType)
{
    int actualCount = count > 300 ? 300 : count;
    for (int i = 0; i < actualCount; i++) {
        Point3D p = getContainerPoint(containerType);
        TGLSphere* sp =
(TGLSphere*) (dcBlock->AddNewChild(__classid(TGLSphere)));

        float normRadius = (float)(1 + rand() % 100) / 100.0f;
        sp->Radius = normRadius * 20.0f + 5.0f;
        sp->Position->SetPoint(p.x, p.y, p.z);
        sp->Stacks = 12; sp->Slices = 12;

        int colIdx = rand() % 7;
        float cr = RAINBOW_COLORS[colIdx][0];
        float cg = RAINBOW_COLORS[colIdx][1];
        float cb = RAINBOW_COLORS[colIdx][2];

        sp->Material->FrontProperties->Diffuse->Color = VectorMake(cr, cg,
cb, 1.0f);
        sp->Material->FrontProperties->Emission->Color = VectorMake(cr *
0.4f, cg * 0.4f, cb * 0.4f, 1.0f);
        m_spheres.push_back(sp);
    }
}
```

Конец листинга 3

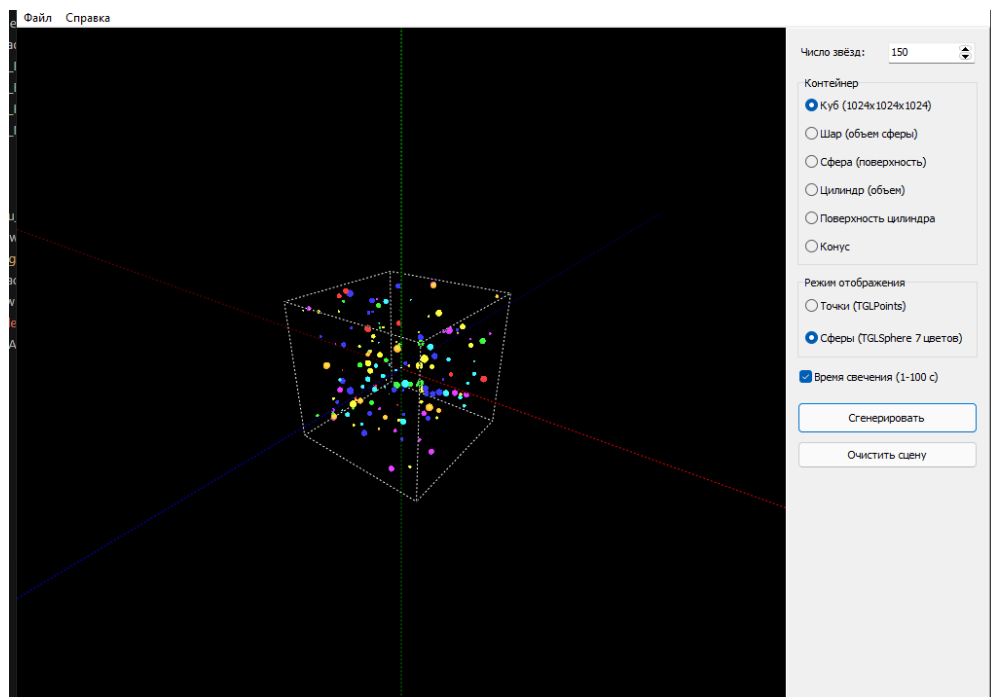


Рисунок 8 – Визуализация спектрального ансамбля 3D-сфер радужных цветов в кубическом контейнере.

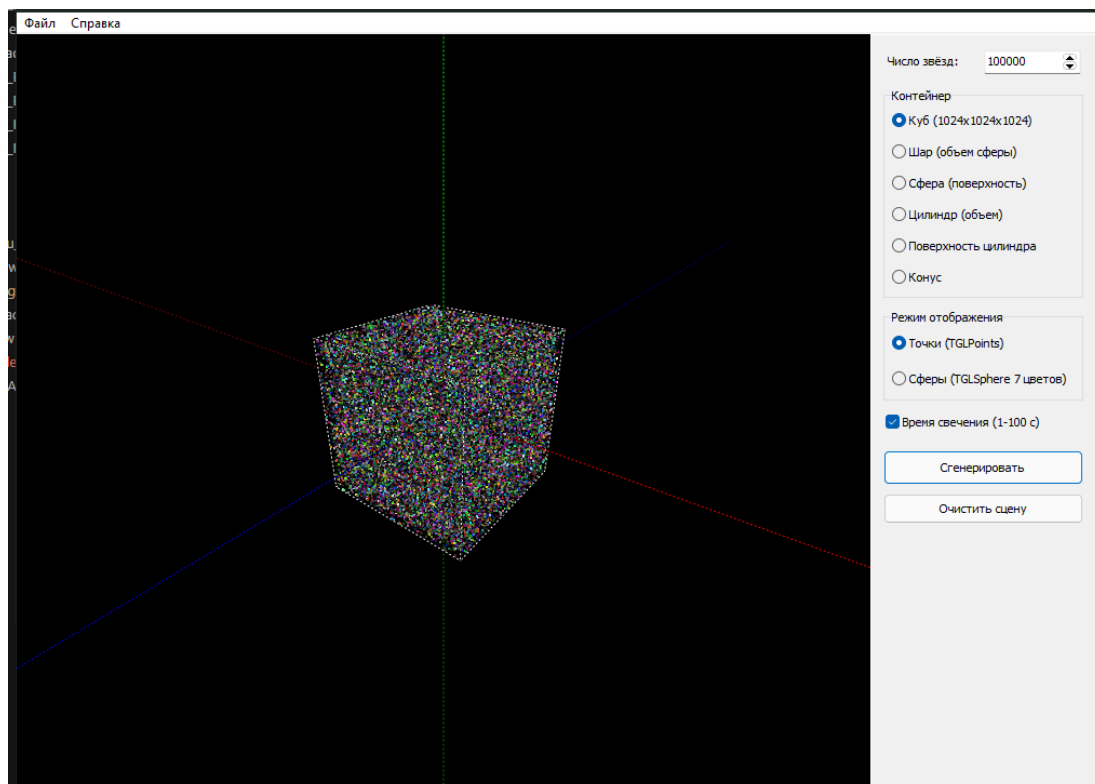


Рисунок 9 - Полигональная сферическая модель звёзд со спектральной раскраской КОЖЗГСФ

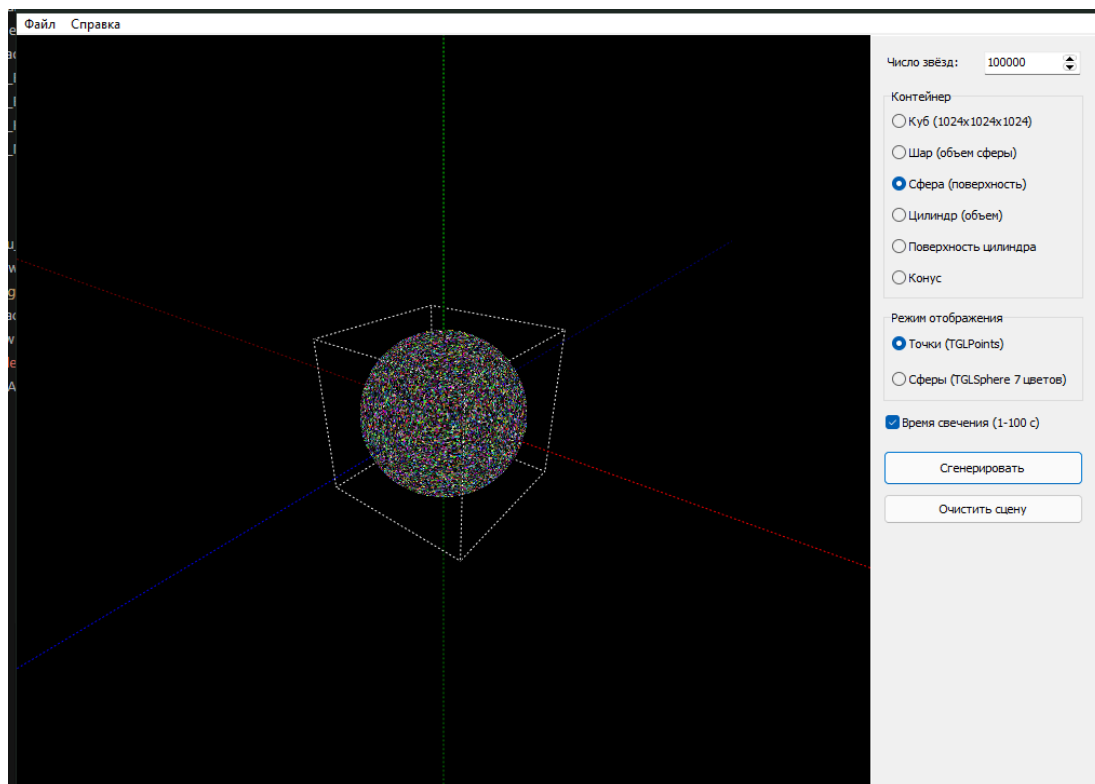


Рисунок 10 – Размещение полигональных разноцветных сфер в коническом объеме.

В строке состояния StatusBar приложения отображается контрольная диагностическая информация: затраченное процессорное время на расчет координат и актуальная частота отрисовки сцены (FramesPerSecond). Для точечного массива из 100 000 элементов средняя частота кадров составляет 230–255 FPS при времени генерации менее 0.09 с. В режиме полигональных сфер со сложными оптическими материалами кадровая частота превышает 1200 FPS, что доказывает высокую производительность конвейера GLScene.

Вывод: В рамках практического исследования были решены ключевые задачи трехмерного геометрического моделирования в библиотеке GLScene.